

Phase 3: Context Engineering & Tool Integration

POC-07 — Inventory Management & Procurement System

Phase Weight: 20% | **Duration:** 5 working days | **Test Cases:** 20

1. Phase Overview

Build a LangChain ReAct agent that answers inventory queries by calling Phase 1 REST API endpoints as tools. The agent can handle questions like "Which grocery products are below reorder point?" and "What is the total value of our current inventory?"

2. Tool Definitions (agent/tools.py)

```
import requests
import structlog
from langchain.tools import tool
from opentelemetry import trace

logger = structlog.get_logger()
tracer = trace.get_tracer("poc-07-agent")
BASE_URL = "http://localhost:8000/api/v1"

def _api_get(path: str, params: dict = None) -> dict:
    try:
        r = requests.get(f"{BASE_URL}{path}", params=params or {}, timeout=10)
        r.raise_for_status()
        return r.json()
    except requests.exceptions.ConnectionError:
        return {"error": "API unavailable"}
    except requests.exceptions.HTTPError as e:
        if e.response.status_code == 404:
            return {"error": "not_found"}
        return {"error": str(e)}
    except Exception as e:
        return {"error": str(e)}

@tool("get_low_stock_alerts")
def get_low_stock_alerts(query: str = "") -> str:
    """Use when asked about low stock products, out-of-stock items, products
    needing reorder,
    or inventory alerts. Returns all products with quantity_available at or below
    their reorder
    point, including out-of-stock items. Essential for procurement
    prioritization."""
    with tracer.start_as_current_span("tool.get_low_stock_alerts") as span:
        span.set_attribute("poc_id", "POC-07")
        data = _api_get("/stock/low-alerts")
```

```

        if "error" in data:
            return f"Error: {data['error']}"
        if not data:
            return "No low stock alerts. All products are above reorder points."
        lines = [f"- {p['sku']}: {p['name']} - {p['quantity_available']} available\n"
                  (reorder at {p['reorder_point']})"]
        for p in data[:20]:
            logger.info("tool_called", poc_id="POC-07", phase="P3",
                        tool="get_low_stock_alerts")
            return f"{len(data)} products need reorder:\n" + "\n".join(lines)

@tool("get_product_stock")
def get_product_stock(product_id: str) -> str:
    """Use when asked about a specific product's stock level, quantity on hand,
    availability, or recent movements. Returns product details with current stock
    quantities and reorder configuration."""
    with tracer.start_as_current_span("tool.get_product_stock") as span:
        span.set_attribute("poc_id", "POC-07")
        data = _api_get(f"/products/{product_id}")
        if "error" in data:
            return f"Product {product_id} not found." if "not_found" in
str(data.get("error")) \
                else f"Error: {data['error']}"
            stock = data.get("stock_level", {})
            logger.info("tool_called", poc_id="POC-07", phase="P3",
                        tool="get_product_stock")
            return (f"Product: {data.get('sku')} - {data.get('name')}\n"
                    f"  On hand: {stock.get('quantity_on_hand', 0)}, "
                    f"Available: {stock.get('quantity_available', 0)}, "
                    f"Reserved: {stock.get('quantity_reserved', 0)}\n"
                    f"  Reorder point: {data.get('reorder_point')}, "
                    f"Reorder qty: {data.get('reorder_quantity')}")

@tool("get_supplier_catalog")
def get_supplier_catalog(supplier_id: str) -> str:
    """Use when asked about what products a supplier carries, supplier pricing, or
    which products to include in a purchase order for a specific supplier. Returns
    the supplier's catalog with product names, SKUs, and cost prices."""
    with tracer.start_as_current_span("tool.get_supplier_catalog") as span:
        span.set_attribute("poc_id", "POC-07")
        data = _api_get(f"/suppliers/{supplier_id}/catalog")
        if "error" in data:
            return f"Supplier {supplier_id} catalog unavailable."
        if not data:
            return f"Supplier {supplier_id} has no products in catalog."
        lines = [f"- {p['sku']}: {p['name']} - ₹
{p['cost_price']:.2f}/{p['unit_of_measure']}"
                  for p in data[:20]]
        logger.info("tool_called", poc_id="POC-07", phase="P3",
                    tool="get_supplier_catalog")
        return f"Supplier {supplier_id} catalog ({len(data)} products):\n" +
"\n".join(lines)

@tool("get_dashboard_stats")

```

```

def get_dashboard_stats(query: str = "") -> str:
    """Use when asked for an overall inventory summary, total product count, how
    many
    items are low stock or out of stock, open purchase orders, or total inventory
    value.
    Returns the inventory management dashboard metrics."""
    with tracer.start_as_current_span("tool.get_dashboard_stats") as span:
        span.set_attribute("poc_id", "POC-07")
        data = _api_get("/dashboard")
        if "error" in data:
            return f"Dashboard unavailable: {data['error']}"
        logger.info("tool_called", poc_id="POC-07", phase="P3",
tool="get_dashboard_stats")
        return (f"Inventory Dashboard:\n"
                f"  Total products: {data.get('total_products', 0)}\n"
                f"  Low stock: {data.get('low_stock_count', 0)}\n"
                f"  Out of stock: {data.get('out_of_stock_count', 0)}\n"
                f"  Open POs: {data.get('open_po_count', 0)}\n"
                f"  Total stock value: ₹{data.get('total_stock_value', 0):.2f}")

@tool("search_inventory_policy")
def search_inventory_policy(question: str) -> str:
    """Use when asked about inventory management rules, reorder point calculation,
    PO process, stock movement types, supplier management policies, stock count
    procedures, or inventory best practices. Searches the operations manual."""
    with tracer.start_as_current_span("tool.search_inventory_policy") as span:
        span.set_attribute("poc_id", "POC-07")
        try:
            from rag.rag_chain import build_rag_chain, ask_question
            result = ask_question(question, build_rag_chain())
            logger.info("tool_called", poc_id="POC-07", phase="P3",
tool="search_inventory_policy")
            return result.get("answer", "No answer found.")
        except Exception as e:
            return f"Policy search error: {str(e)}"

```

3. System Prompt (agent/prompts.py)

```

INVENTORY_AGENT_SYSTEM_PROMPT = """You are an intelligent inventory management
assistant for a
retail operations platform (POC-07). You help store managers, inventory analysts,
procurement
officers, and warehouse staff manage stock levels and procurement.

```

You have access to these tools:

- get_low_stock_alerts: Get all products at or below reorder point
- get_product_stock: Get stock details for a specific product
- get_supplier_catalog: View products and prices from a supplier
- get_dashboard_stats: Get inventory health summary and totals
- search_inventory_policy: Search operations manual for rules and procedures

Key domain knowledge:

- SKU format: SKU-`{CATEGORY_PREFIX}`-`{NNNN}` (GRO=Grocery, ELC=Electronics, CLO=Clothing)
- PO format: PO-`{YEAR}`-`{NNNN}`
- Low stock: `quantity_available ≤ reorder_point` → alert created
- Out of stock: `quantity_available = 0` → critical alert
- PO lifecycle: draft → submitted → acknowledged → received

Guidelines:

1. For stock level questions about a specific product, use `get_product_stock`
2. For "what needs reordering" questions, use `get_low_stock_alerts`
3. For policy questions, use `search_inventory_policy`
4. Always mention quantities with units (pieces, kg, etc.)
5. Flag out-of-stock situations as urgent""

4. Submission Checklist

- ☐ 5 tools: `get_low_stock_alerts`, `get_product_stock`, `get_supplier_catalog`, `get_dashboard_stats`, `search_inventory_policy`
- ☐ `INVENTORY_AGENT_SYSTEM_PROMPT` references all 5 tools
- ☐ `_api_get` returns `{"error": ...}` on `ConnectionError`
- ☐ All tools registered in executor
- ☐ LangSmith project "AI-Readiness-POC-07-P3" traces
- ☐ 20 test cases: ≥ 14 passing